

Implementación de una página web básica utilizando HTML

Construir una página web básica requiere conocer tanto el **entorno de desarrollo** adecuado como los **fundamentos del lenguaje HTML**. A continuación, se describen detalladamente los pasos y conceptos clave para implementar una página web simple: desde la preparación del entorno (instalación de un editor de texto y uso del navegador) hasta la estructura y etiquetas principales de HTML, incluyendo el uso de recursos (assets), elementos semánticos, accesibilidad y SEO, **lo más completo posible**.

El entorno de desarrollo para HTML

Para desarrollar en HTML es recomendable contar con un *editor de código* especializado y un navegador web con herramientas para desarrolladores. Un buen editor facilita la escritura del código, mientras que el navegador permite probar y depurar la página.

Editor de código (Visual Studio Code u otros)

Aunque cualquier editor de texto plano (como el Bloc de Notas en Windows o TextEdit en Mac) puede servir para escribir código HTML, es preferible usar un **editor de código** dedicado que ofrezca funcionalidades avanzadas. Por ejemplo, **Visual Studio Code (VS Code)** es un editor de código multiplataforma y gratuito altamente recomendado ¹. Para utilizarlo, basta con descargarlo de su sitio oficial e instalarlo siguiendo las instrucciones para Windows, macOS o Linux.

Un editor como VS Code proporciona múltiples ventajas para el desarrollo web:

- **Resaltado de sintaxis:** El código HTML y otros lenguajes se muestran con colores según su sintaxis, lo que facilita la lectura y detección de errores ². Por ejemplo, las etiquetas, atributos y valores pueden aparecer en colores distintos, ayudando a diferenciar cada parte.
- **Autocompletado y sugerencias:** A medida que se escriben etiquetas o atributos, el editor puede sugerir automáticamente completarlos (e incluso cerrar etiquetas HTML de forma automática), ahorrando tiempo y reduciendo errores de sintaxis.
- **Indentación y formato automático:** Los editores suelen manejar la indentación (sangrado) del código de forma consistente, lo que mejora la legibilidad. También suelen resaltar pares de etiquetas o paréntesis coincidentes, facilitando ver dónde termina cada bloque de código ³.
- **Detección de errores básicos:** Si hay un error como una etiqueta sin cerrar o mal escrita, el editor puede subrayarlo o marcarlo, ayudando a detectarlo rápidamente. Por ejemplo, VS Code puede resaltar en rojo una etiqueta `<p>` sin su `</p>` de cierre.
- **Extensiones y funcionalidades extra:** Editores modernos permiten instalar *extensiones* para agregar funcionalidades. En desarrollo web son útiles extensiones como *Live Server* (que actualiza la página en el navegador al guardar cambios) o *Emmet* (abreviaturas que expanden en código HTML completo), entre otras.

En resumen, un buen editor de código agiliza el desarrollo al brindar soporte y herramientas enfocadas a código, a diferencia de un procesador de textos común que no está pensado para manejar HTML ⁴.

Inspector de elementos del navegador (DevTools)

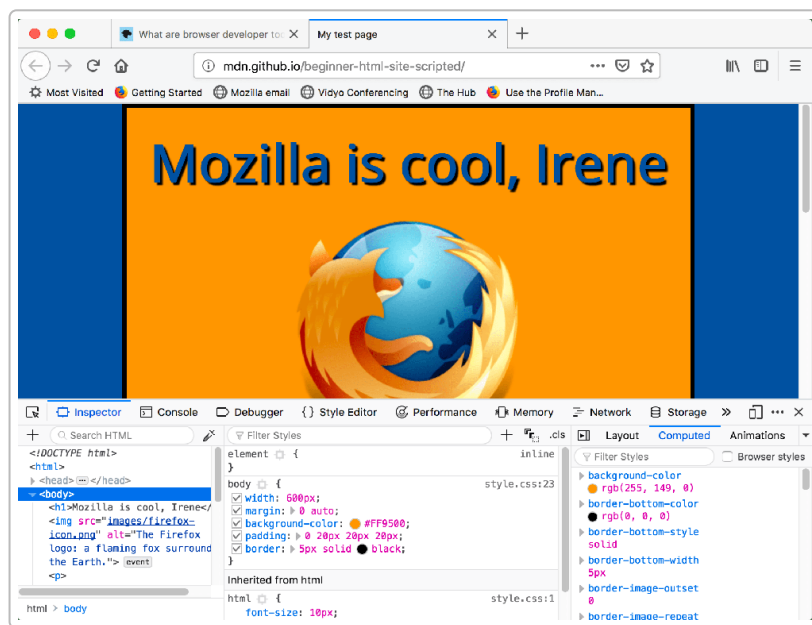


Figura: Herramientas de desarrollo (DevTools) con la pestaña Inspector abierta, mostrando el código HTML y los estilos CSS de una página web.

Todos los navegadores web modernos (Chrome, Firefox, Edge, Safari, etc.) incluyen un conjunto de **herramientas para desarrolladores** (*Developer Tools* o *DevTools*). Estas herramientas permiten inspeccionar el código **HTML**, las **CSS** y el **JavaScript** cargados en la página, así como otros aspectos como los recursos (imágenes, archivos) y el rendimiento de carga ⁵. En particular, la sección de **Inspeccionar Elementos** (Inspector) muestra la estructura del DOM (el árbol de elementos HTML) tal como el navegador la interpreta, y permite ver e incluso editar temporalmente el HTML y CSS en tiempo real.

Usar el inspector es fundamental para depurar y verificar el correcto funcionamiento de la página. Por ejemplo, si algún elemento no se ve como esperabas, puedes hacer clic derecho sobre él en la página y elegir **"Inspeccionar"** (o usar el atajo **Ctrl+Shift+I** en Windows/Linux, **Cmd+Option+I** en Mac) para abrir el panel Inspector directamente en ese elemento. Verás el código HTML correspondiente resaltado en la vista del DOM, y en un panel adjunto los estilos CSS aplicados. Esto te permite comprobar la estructura: por ejemplo, si olvidaste cerrar una etiqueta, el DOM mostrado puede revelar una anidación incorrecta de elementos. También puedes editar el HTML desde esta herramienta (por ejemplo, agregar manualmente un `</p>` que falte) y ver instantáneamente cómo cambiaría la página, lo cual es útil para identificar y solucionar el problema ⁶ ⁷. Ten en cuenta que los cambios hechos en DevTools **no** se guardan en tu archivo, solo sirven para pruebas en ese momento.

Además, las DevTools incluyen una **Consola** donde se registran errores o mensajes (por ejemplo, si una imagen no se encontró, si hay errores de JavaScript, etc.), y pestañas de **Red (Network)** para ver si todos los archivos se cargaron correctamente. En conjunto, estas herramientas te ayudan a **detectar inconsistencias** o errores de la página web y verificar su correcto funcionamiento, desde la estructura HTML hasta la carga de assets y la aplicación de estilos.

Fundamentos del lenguaje HTML

HTML, siglas de **HyperText Markup Language** (Lenguaje de Marcado de Hipertexto), es el lenguaje estándar para estructurar el contenido de las páginas web. A diferencia de un lenguaje de programación, HTML es un **lenguaje de marcado**, lo que significa que emplea **etiquetas (tags)** para delimitar elementos dentro del contenido y darles un significado o rol específico en la página ⁸. En otras palabras, HTML consiste en una serie de elementos que *marcan* partes del texto (o contenido multimedia) indicando, por ejemplo, cuándo algo es un párrafo, un encabezado, un enlace, una imagen, etc. Cada elemento HTML generalmente tiene una **etiqueta de apertura** `<...>` y una **etiqueta de cierre** `</...>` que envuelven al contenido, definiendo así su alcance. Por ejemplo, si se tiene el texto plano `Hola mundo`, al envolverlo en la etiqueta de párrafo quedaría `<p>Hola mundo</p>`, indicando al navegador que ese texto es un párrafo. Del mismo modo, `<strong>importante</strong>` marcaría una palabra como importante (generalmente se muestra en negrita), y `<a href="https://example.com">enlace</a>` convertiría un texto en un hipervínculo apuntando a otra URL.

Estructura básica de un documento HTML

Toda página web HTML requiere una estructura mínima para que el navegador interprete correctamente el contenido. Esta estructura define secciones principales del documento:

- `<!DOCTYPE html>` – Debe ubicarse al inicio del archivo. Es la declaración del tipo de documento, que en HTML5 se escribe simplemente así (indica al navegador que el documento sigue el estándar HTML5) ⁹.
- `<html> ... </html>` – Encierra todo el contenido de la página web. La etiqueta `<html>` al comienzo y su `</html>` al final le indican al navegador dónde inicia y termina el documento HTML ¹⁰. A la etiqueta `<html>` se le puede añadir el atributo `lang` para indicar el idioma del contenido (por ejemplo `<html lang="es">` para español).
- `<head> ... </head>` – Dentro de `<html>`, la sección `<head>` agrupa metadatos e información **no visible directamente** en la página, pero importante para el documento ¹¹. Aquí se incluye el **título** de la página (mediante `<title>`), definiciones de **codificación de caracteres** (p.ej. `<meta charset="UTF-8">` para indicar que se usa UTF-8), metadatos de SEO (descripción, palabras clave, autores), enlaces a **hojas de estilo CSS** externas (`<link rel="stylesheet" href="styles.css">`), enlaces a íconos (favicon) y eventualmente scripts que se quieran cargar en la cabecera. El contenido del `<head>` no se muestra en la ventana del navegador, pero sí influye en cosas como el título que aparece en la pestaña del navegador, la forma en que se procesa el texto, etc.
- `<body> ... </body>` – También dentro de `<html>`, la sección `<body>` contiene el **contenido visible** de la página ¹². Todo texto, imagen, enlace o elemento que deba mostrarse al usuario en la página debe estar dentro del `<body>`. Es la parte "cuerpo" del documento donde estructuramos el contenido utilizando las diversas etiquetas HTML disponibles.

Estas tres secciones fundamentales (`html`, `head` y `body`) conforman el esqueleto de cualquier página HTML. A continuación, se muestra un ejemplo mínimo de documento HTML con esta estructura básica:

```
<!DOCTYPE html>
<html lang="es">
  <head>
    <meta charset="UTF-8" />
```

```
<title>Mi primera página</title>
</head>
<body>
  <h1>¡Hola mundo!</h1>
  <p>Esta es mi primera página web básica con HTML.</p>
</body>
</html>
```

En el ejemplo anterior, se define el doctype de HTML5, luego la raíz `<html lang="es">` con el idioma, dentro de ella el `<head>` que especifica la codificación UTF-8 y el título de la página ("Mi primera página"), y finalmente el `<body>` con un encabezado de nivel 1 (h1) y un párrafo de texto.

Una vez que se tiene este código, se puede **guardar en un archivo con extensión .html** (por convención, a la página principal suele llamársele `index.html`). Es importante guardar el archivo en formato **texto plano UTF-8**. El nombre del archivo debería escribirse en minúsculas, sin espacios ni caracteres especiales, idealmente ¹³. Por ejemplo: `index.html` es un nombre válido y común. Tras guardar, simplemente abre el archivo en un navegador (doble clic o arrastrar y soltar en la ventana del navegador).

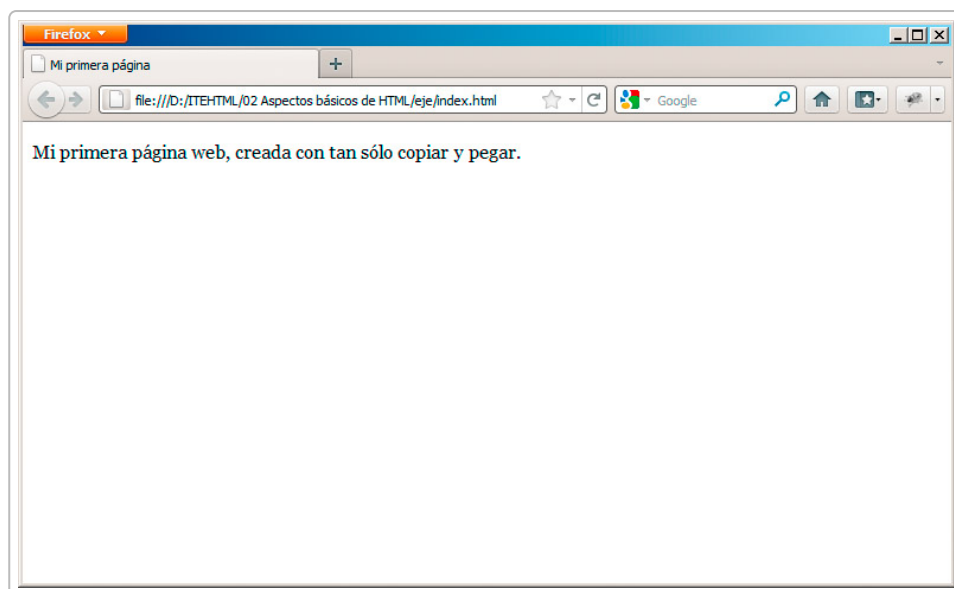


Figura: Visualización en el navegador del documento HTML de ejemplo. El contenido de `<body>` ("Mi primera página web...") se muestra en la ventana, y el texto de `<title>` ("Mi primera página") aparece en la pestaña del navegador.

Al abrir el archivo HTML en el navegador, verás que el título definido en `<title>` aparece en la pestaña o barra superior del navegador, mientras que el contenido del `<body>` (encabezados, párrafos, etc.) aparece en la ventana principal ¹⁴. En nuestro ejemplo, el navegador mostraría un gran encabezado "¡Hola mundo!" seguido del texto del párrafo, y en la pestaña se leería "Mi primera página". Si todo se configuró correctamente (como la metaetiqueta de charset), caracteres especiales como tildes o eñes deberían verse sin problema. En este punto, ¡ya tendrías tu primera página web estática funcionando!

Etiquetas HTML principales y atributos

Dentro del `<body>` podemos utilizar **múltiples etiquetas HTML** para dar forma al contenido de la página. Algunas de las etiquetas **más comunes** para estructurar contenido son:

- **Encabezados** (`<h1>` ... `<h6>`): Sirven para definir títulos y subtítulos. `<h1>` indica un encabezado principal (el más importante, generalmente único por página), `<h2>` un subtítulo de segundo nivel, y así sucesivamente hasta `<h6>` para un encabezado de sexto nivel. Estas etiquetas estructuran jerárquicamente la información.
- **Párrafo** (`<p>`): Define un párrafo de texto normal. Se utiliza para agrupar oraciones en bloques de texto separados por un espacio vertical (salto de línea automático antes y después).
- **Enlace** (`<a>`): Crea un hipervínculo hacia otra URL o hacia otra parte de la misma página. Requiere usar el atributo `href` para especificar la dirección. Por ejemplo:
`<a href="https://www.ejemplo.com">Visitar sitio</a>` crea un enlace clickable.
(Texto ancla visible: "Visitar sitio").
- **Imagen** (`<img>`): Inserta una imagen en la página. Es una etiqueta *vacía* (no tiene cierre y no encierra contenido textual). Debe incluir al menos el atributo `src="ruta_del_archivo"` para indicar la ubicación del archivo de imagen, y **siempre** un atributo `alt="descripción"` que proporciona un texto alternativo descriptivo de la imagen (usado para accesibilidad y mostrado si la imagen no carga). Por ejemplo:
``. Opcionalmente se pueden especificar atributos de dimensión (`width`, `height`) u otros.
- **Lista desordenada** (`<ul>`): Crea una lista de ítems sin un orden numérico (por defecto aparece con viñetas). Cada elemento de la lista se define con `<li>` (list item). Por ejemplo:

```
<ul>
  <li>Elemento 1</li>
  <li>Elemento 2</li>
  <li>Elemento 3</li>
</ul>
```

definirá una lista con tres ítems con viñetas.

- **Lista ordenada** (`<ol>`): Similar a `<ul>` pero genera una lista numerada (1, 2, 3, ...). También utiliza `<li>` para cada elemento.
- **Tabla** (`<table>`): Permite presentar datos tabulares en filas y columnas. Dentro de `<table>` se definen filas con `<tr>` (table row) y, dentro de cada fila, celdas de datos con `<td>` (table data). Por ejemplo, una tabla mínima:

```
<table>
  <tr>
    <td>Celda 1</td>
    <td>Celda 2</td>
  </tr>
  <tr>
    <td>Celda 3</td>
    <td>Celda 4</td>
  </tr>
</table>
```

Las tablas se usan *solo* para datos estructurados en celdas; **no** deben usarse para maquetar el diseño de la página (para eso se usan CSS), ya que un abuso de tablas dificulta la accesibilidad y mantenimiento.

(Por supuesto, HTML ofrece muchas más etiquetas - por ejemplo `<br>` para saltos de línea manuales, `<span>` para marcar texto en línea sin significado específico, `<div>` para agrupar bloques, `<form>` para formularios, etc. Pero nos centramos en las esenciales de contenido.)

Notemos que las etiquetas HTML pueden llevar **atributos** que aportan información adicional o modifican el comportamiento por defecto del elemento. Ya mencionamos algunos ejemplos arriba, como `href` en los enlaces, `src` y `alt` (y dimensiones) en las imágenes. En general, un atributo se coloca **dentro de la etiqueta de apertura** de un elemento, con la sintaxis `nombre="valor"`. Por ejemplo, en `<p class="destacado">Texto</p>`, el atributo `class="destacado"` le asigna al párrafo una clase CSS llamada "destacado" (que podría usarse luego para aplicar estilos CSS específicos a ese párrafo). Un mismo elemento puede tener varios atributos. Siempre se debe respetar el formato: nombre del atributo en minúsculas, seguido de `=`, y el valor entre comillas ¹⁵. Algunos atributos booleanos (como `checked` en inputs tipo checkbox) pueden a veces usarse sin valor, pero por consistencia y compatibilidad es mejor asignarles `= "checked"` aunque sea redundante.

Importante: Ciertos atributos son obligatorios en elementos específicos. Por ejemplo, `alt` es obligatorio en `<img>`, ya que provee el texto alternativo de la imagen por accesibilidad; si no se proporciona, se considera mala práctica (y algunas herramientas de validación lo marcarán como error). Del mismo modo, un enlace `<a>` debe tener `href` (a menos que sea usado solo como ancla interna con `id`). Estos atributos aseguran que el elemento cumpla su función adecuadamente.

Uso de recursos (assets) y rutas relativas

En una página HTML es común incluir **recursos externos** como imágenes, hojas de estilo CSS o archivos JavaScript. Para referenciar estos archivos dentro de nuestro HTML, utilizamos rutas (paths). Es fundamental entender la diferencia entre **rutas relativas** y **rutas absolutas** para gestionar correctamente estos assets:

- **Ruta relativa:** indica la ubicación de un archivo **en relación al documento HTML actual**, sin especificar el dominio ¹⁶. Por ejemplo, supongamos que tenemos una carpeta `imagenes` al lado de nuestra página HTML, que contiene `foto.png`. Si queremos insertar esa imagen en la página, usaríamos una ruta relativa en el atributo `src`: ``. Esta ruta asume que la carpeta `imagenes` está en el mismo directorio (o un subdirectorio) respecto al archivo HTML donde se escribe. Las rutas relativas **no llevan** `http://` ni dominio; pueden usar `../` para subir niveles de carpeta si es necesario. Otro ejemplo: `<a href="contacto/info.html">Más información</a>` asume que desde la ubicación del HTML actual, existe una carpeta `contacto` con un archivo `info.html` dentro. Si movemos todo el proyecto web a otro dominio o a otra ruta, las rutas relativas siguen funcionando siempre que se conserve la misma estructura de carpetas.

- **Ruta absoluta:** especifica la **ubicación completa** de un recurso, incluyendo el dominio o la raíz del sitio. Por lo general comienza con `http://` o `https://` seguido del dominio, o con una `/` indicando la raíz del servidor actual. Por ejemplo, una ruta absoluta para la misma imagen sería `` (incluyendo dominio), o bien `/imagenes/foto.png` si se refiere a la raíz del mismo sitio. Las rutas absolutas se utilizan principalmente para enlazar recursos que están en **otro sitio o**

servidor (por ejemplo, una biblioteca CSS/JS de un CDN, o una imagen alojada en otro dominio) ¹⁷. También se pueden usar para enlaces internos, pero **no se recomienda** abusar de rutas absolutas locales: si en el futuro migramos el sitio a otro dominio, todos esos enlaces absolutos locales dejarían de funcionar al apuntar al dominio antiguo ¹⁸. En desarrollo, las rutas absolutas dificultan probar el sitio en entornos distintos (por ejemplo, abrir el archivo localmente mostrará rutas rotas si esperan un dominio). Por ello, **para recursos internos de nuestro propio sitio, es preferible usar rutas relativas** en la mayoría de los casos ¹⁹, ya que aportan portabilidad y flexibilidad (podemos mover el sitio de carpeta, de servidor o de dominio sin tener que cambiar todos los enlaces). Las rutas absolutas las reservaremos para casos necesarios: referenciar recursos alojados externamente o situaciones especiales de SEO (como la etiqueta `<link rel="canonical">` que sí requiere URL completa).

En resumen, si nuestros archivos (imágenes, CSS, etc.) están dentro de nuestro proyecto web, especificaremos la ruta partiendo de la ubicación del HTML actual. Si estamos apuntando a un recurso de otro sitio o a una ubicación fija del nuestro (independiente de la página actual), usaremos ruta absoluta. Por ejemplo, para enlazar una hoja de estilo CSS que tenemos en una carpeta `css` podríamos usar: `<link rel="stylesheet" href="css/estilos.css">` (ruta relativa a la carpeta `css`), mientras que para incluir un script de un CDN usaríamos algo como `<script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>` (ruta absoluta completa a ese recurso externo).

Nota: Es buena práctica también organizar nuestros assets en carpetas (por ejemplo, `css` para estilos, `js` para scripts, `images` para imágenes) y utilizar rutas relativas coherentes. Asimismo, verificar en las herramientas de desarrollador (pestaña "Network" o "Red") que todos los recursos carguen (código 200 OK) y no haya errores 404, especialmente al desplegar el sitio en un servidor – esto ayuda a detectar rutas mal escritas.

Etiquetas semánticas, accesibilidad y SEO

Con HTML5 se introdujeron las **etiquetas semánticas**: elementos que, más allá de solo definir un bloque, *describen* el rol de ese bloque de contenido dentro de la página. Antes de HTML5, muchos desarrolladores estructuraban secciones usando genéricamente `<div>` (que no aporta significado por sí mismo). Ahora contamos con etiquetas como `<header>`, `<nav>`, `<main>`, `<section>`, `<article>`, `<aside>` y `<footer>` que indican claramente secciones con roles específicos. Estas etiquetas **semánticas** hacen que el código sea más **legible** y aportan significado tanto para los desarrolladores como para los agentes de usuario (navegadores, motores de búsqueda, lectores de pantalla, etc.) ²⁰. Por ejemplo, un `<header>` típico contendrá el encabezado de la página o de una sección (logo, título, menú principal), `<nav>` contendrá un bloque de enlaces de navegación, `<main>` envolverá el contenido principal único de la página, `<article>` puede usarse para un artículo o entrada independiente, `<section>` para agrupar contenido relacionado dentro de una página, `<aside>` para contenido complementario (como barras laterales, bloques de información relacionada) y `<footer>` para el pie de página con información de cierre o enlaces de contacto ²¹.

²². A continuación se listan estas etiquetas semánticas estructurales principales y su función:

- `<header>` – Encabezado de la página o de una sección. Suele incluir el título del sitio o sección, logotipo, menú principal de navegación, cabeceras introductorias, etc.
- `<nav>` – Sección de navegación. Agrupa usualmente un conjunto de enlaces de navegación (por ejemplo, el menú del sitio). Puede haber varios `<nav>` si hay navegaciones secundarias (ej.: un menú de categorías aparte del menú principal).

- `<main>` – Contenido principal de la página. Debe haber solo un `<main>` por documento, y contendrá la información central única de esa página (lo más relevante de ese URL). Los lectores de pantalla pueden saltar directamente al `<main>`.
- `<article>` – Representa un bloque de contenido autónomo e independiente, como un artículo, entrada de blog, comentario, tarjeta de producto, etc. Puede concebirse como una unidad que tiene sentido por sí misma (y podría, por ejemplo, republicarse independiente).
- `<section>` – Define una sección genérica del documento, agrupando contenido temáticamente relacionado. Se usa para dividir el contenido en secciones lógicas, normalmente con un encabezado propio. A diferencia de `<article>`, una `<section>` normalmente forma parte de algo más grande (por ejemplo, secciones dentro de un artículo) y no necesariamente es independiente por sí sola.
- `<aside>` – Sección de contenido "al margen" del contenido principal. Suele emplearse para barras laterales, cuadros de información extra, anuncios, enlaces relacionados, etc. Es contenido relevante pero **complementario**; si no existiera, la página principal seguiría teniendo sentido.
- `<footer>` – Pie de página de la página o de una sección. Incluye información de cierre: típicamente información de contacto, derechos de autor, enlaces a términos de uso, o navegación secundaria. Suele ir al final del contenido principal o de una sección cuyo footer complementa.

Usar estas etiquetas semánticas en lugar de elementos genéricos (`<div>`) tiene varias ventajas. En términos de **accesibilidad**, proporciona estructura adicional que las tecnologías de asistencia (como los lectores de pantalla) pueden aprovechar para dar contexto a los usuarios. Por ejemplo, un lector de pantalla puede listar las secciones de la página y decir "Navegación, encabezado principal, contenido principal, barra lateral, pie de página", facilitando la comprensión de la página a usuarios ciegos. De hecho, los elementos semánticos HTML5 ofrecen *pistas adicionales* a los lectores de pantalla sobre el contenido que están navegando ²³. En términos de **SEO (Optimización para motores de búsqueda)**, el HTML semántico ayuda a los *crawlers* (rastreadores como Googlebot) a entender mejor la estructura y relevancia del contenido ²⁴. Por ejemplo, Google puede identificar rápidamente cuál es el encabezado principal de tu página si está envuelto en `<header>` y `<h1>`, o saber qué enlaces conforman el menú de navegación si están dentro de `<nav>`. Esto potencialmente mejora la interpretabilidad de tu página y podría favorecer su posicionamiento para las palabras clave adecuadas (aunque el HTML semántico por sí solo no garantiza un mejor ranking, sí sienta una base técnicamente sólida).

Además de las etiquetas de sección mencionadas, HTML ofrece etiquetas semánticas a nivel de texto, como `<strong>` para indicar énfasis fuerte (generalmente mostrado en negrita) o `<em>` para énfasis sutil (generalmente cursiva), `<blockquote>` para citas en bloque, `<q>` para citas en línea, `<code>` para código, `<address>` para información de contacto, `<time>` para marcar fechas u horas, etc. Estas etiquetas aportan significado al texto más allá de la apariencia visual (por ejemplo, `<strong>` significa semánticamente "esto es importante", no solo "esto está en negrita"). Siempre que sea posible, es preferible usar una etiqueta semántica adecuada en lugar de una genérica; por ejemplo, usar `<em>` en lugar de `<i>` solo para cursiva sin significado, o `<strong>` en lugar de `<b>` para negrita sin significado.

Buenas prácticas de semántica y accesibilidad: Al emplear HTML semántico, es importante seguir una estructura lógica. Por ejemplo, organizar los encabezados por niveles de importancia, de forma anidada: un `<h2>` debe estar debajo de un `<h1>` como subtema, y si hay sub-subtemas, usar `<h3>` dentro de esa sección, y así sucesivamente. Mantener esta jerarquía de encabezados ayuda tanto a la **accesibilidad** (los lectores de pantalla pueden anunciar la estructura jerárquica y permitir saltar de sección en sección) como al **SEO**, ya que refuerza la comprensión temática del contenido ²⁵. No saltes niveles arbitrariamente (por ejemplo, no uses un `<h1>` para cada elemento destacando solo por tamaño; generalmente se usa un único `<h1>` por página para el título principal, luego `<h2>` para

secciones principales, etc.). Recuerda que los estilos visuales (tamaño de texto, colores) deben manejarse con CSS; no utilices mal las etiquetas semánticas solo para lograr un cierto aspecto visual ²⁶.

Por último, la **accesibilidad web** también implica proveer alternativas y atributos adecuados: siempre incluir textos alternativos con significado en las imágenes mediante `alt` (por ejemplo, ``), de modo que usuarios con discapacidad visual sepan qué imagen está presente ²⁷. Si la imagen es puramente decorativa, el atributo `alt` puede dejarse vacío (`alt=""`) pero nunca omitir el atributo por completo. Para elementos multimedia como audio/vídeo, proveer transcripciones o subtítulos cuando corresponda. Utilizar etiquetas de formulario (`<label>` para los campos de input, etc.) para asegurar que cada control esté identificado. Todo esto mejora la usabilidad y también la forma en que los buscadores interpretan el contenido (por ejemplo, el atributo `alt` contribuye al SEO en búsquedas de imágenes y mejora la semántica general de la página).

En síntesis, **HTML semántico** significa usar la etiqueta correcta para el propósito correcto: esto produce un código más claro, accesible y con beneficios de SEO. Al implementar una página básica, aplicar estos principios desde el inicio (estructura clara con `<header>`, `<nav>`, `<main>`, encabezados ordenados, texto alternativo en imágenes, etc.) sienta una base sólida para un sitio web profesional y amigable tanto para usuarios como para motores de búsqueda ²⁸.

Fuentes utilizadas: Se han consultado documentación y guías reconocidas, como *MDN Web Docs* de Mozilla (conceptos básicos de HTML, editores de código, herramientas de desarrollador, accesibilidad) ^{8 10 5 23}, artículos especializados (ej. Semrush para etiquetas semánticas y SEO) ^{28 21}, y material educativo en español (p. ej. La Nuve Artística, Hostalia) para mejores prácticas de estructuración y rutas en HTML ^{9 17}, entre otros. Estas fuentes respaldan las recomendaciones y explicaciones dadas a lo largo del texto. ¡Con este conocimiento, estás listo para crear y comprobar tu primera página web en HTML de manera correcta y eficiente! ^{7 27}

^{1 2 3 4} Editores de código - Aprende desarrollo web | MDN

https://developer.mozilla.org/es/docs/Learn_web_development/Getting_started/Environment_setup/Code_editors

⁵ ¿Cuáles son las herramientas de desarrollo del navegador? - Aprende desarrollo web | MDN

https://developer.mozilla.org/es/docs/Learn_web_development/Howto/Tools_and_setup/What_are_browser_developer_tools

^{6 7} Inspeccionar elementos en el navegador para encontrar errores

<https://www.eniun.com/inspeccionar-elementos-navegador-encontrar-errores-css-html/>

^{8 15} Conceptos básicos de HTML - Aprende desarrollo web | MDN

https://developer.mozilla.org/es/docs/Learn_web_development/Getting_started/Your_first_website/Creating_the_content

^{9 10 11 12 13 14} Estructura básica de una página | Aspectos básicos del lenguaje HTML

https://www.lanubeartistica.es/Curso_html5/dos/estructura_basica_de_una_pagina.html

^{16 27} Rutas Relativas vs Absolutas en HTML – Guía Completa con Ejemplos

<https://asimov.cloud/blog/html-38/20-rutas-relativas-y-absolutas-en-html-guia-completa-con-ejemplos-practicos-561>

^{17 18 19} Rutas absolutas y relativas: Guía SEO | Hostalia

<https://blog.hostalia.com/hostalia/guia-rutas-absolutas-relativas/>

^{20 21 22 24 25 26 28} HTML Semántico: Guía Avanzada Para Implementarlo

<https://es.semrush.com/blog/html-semantico/>

23 HTML: Una buena base para la accesibilidad - Aprende desarrollo web | MDN

https://developer.mozilla.org/es/docs/Learn_web_development/Core/Accessibility/HTML